Programowalne Układy Cyfrowe – Ćwiczenie 7

Cele:

- zapoznanie z buforami FIFO (First-In First-Out) - kolejka,
- zapoznanie z buforami LIFO (Last-In First-Out) - stos.

Przebieg ćwiczenia:

1. Uruchomić program Active-HDL, stworzyć nowy projekt.
2. W postaci dwóch plików źródłowych wprowadzić następujący program:

Plik testbench.v

```
module mytestbenchmodule();
reg CLK;
initial CLK  <= 0;
always #50  CLK <= ~CLK;

reg RST;
initial
begin
    RST <= 0;
    RST <= #100 1;
    RST <= #500 0;
end

reg fi_stb;
reg [7:0]  fi_dat;
wire fi_bsy;
wire fo_stb;
wire [7:0] fo_dat;
reg fo_ack;

initial
begin
    fi_stb <= 0;
    fo_ack <= 0;
    fi_dat <= 0;
    #1000;
    @(negedge CLK);
    fi_stb <= 1;
    fi_dat <= 17; @(negedge CLK);
    fi_dat <= 18; @(negedge CLK);
    //      !!!!!!!!!!!!!!!! ....  please type some additional data here ... //
    //      !!!!!!!!!!!!!!!! ....  tutaj prosze wpisac dodatkowe dane ... //
    fi_dat <= 55; @(negedge CLK);
    fi_stb <= 0;
    #1000;
    @(negedge CLK);
    fo_ack <= 1;
end

fifo
#(
.WIDTH(8)
)
f1
(
.CLK(CLK),
.RST(RST),
.FI_STB(fi_stb),
.FI_DAT(fi_dat),
.FI_BSY(fi_bsy),
.FO_STB(fo_stb),
.FO_ACK(fo_ack),
.FO_DAT(fo_dat)
);
endmodule
```

Plik fifo.v

```
module fifo
#(
parameter WIDTH = 8
)
(
input  wire        CLK,
input  wire        RST,
input  wire              FI_STB,
input  wire  [WIDTH-1:0] FI_DAT,
output wire              FI_BSY,
output wire              FO_STB,
input  wire              FO_ACK,
output wire  [WIDTH-1:0] FO_DAT
);
//--------------------------------------
```

```
reg   [WIDTH-1:0] ff_dat [0:15];
reg   [3:0] ff_sel;
reg         ff_in_busy;
reg         ff_out_stb;
//--------------------------------------
integer     ff_state;
//--------------------------------------
assign FI_BSY     = ff_in_busy;
//--------------------------------------
assign FO_STB     = ff_out_stb;

//--------------------------------------
always@(posedge CLK or posedge RST) ff_in_busy <= (RST) ? 1'b0 : (ff_state==4);
//--------------------------------------
always@(posedge CLK or posedge RST)
 if(RST)
    begin
            ff_state     <= 0;
            ff_sel       <= 4'hF;
            ff_out_stb   <= 1'b0;
    end
 else casex(ff_state)
//.............................
// clear state
//.............................
0:  begin
     ff_sel      <=                 4'hF;
     ff_out_stb  <=                 1'b0;
     ff_state    <=                    1;
    end
//.............................
// empty
//.............................
1: if(FI_STB)
    begin
     ff_sel      <=         ff_sel + 1'b1;
     ff_out_stb  <=                 1'b1;
     ff_state    <=                    2;
    end
//.............................
// not empty but only one symbol in the buffer
//.............................
2: if(FI_STB && FO_ACK)
    begin
     ff_sel      <=               ff_sel;
     ff_out_stb  <=                 1'b1;
     ff_state    <=                    2;
    end
   else if(FI_STB)
    begin
     ff_sel      <=         ff_sel + 1'b1;
     ff_out_stb  <=                 1'b1;
     ff_state    <=                    3;
    end
   else if(FO_ACK)
    begin
     ff_sel      <=         ff_sel - 1'b1;
     ff_out_stb  <=                 1'b0;
     ff_state    <=                    1;
    end
//.............................
// not empty
//.............................
3: if(FI_STB && FO_ACK)
    begin
     ff_sel      <=               ff_sel;
     ff_out_stb  <=                 1'b1;
     ff_state    <=                    3;
    end
   else if(FI_STB)
    begin
     ff_sel      <=         ff_sel + 1'b1;
     ff_out_stb  <=                 1'b1;
     ff_state    <= (ff_sel>=8)?    4 : 3;
    end
   else if(FO_ACK)
    begin
     ff_sel      <=         ff_sel - 1'b1;
     ff_out_stb  <=                 1'b1;
     ff_state    <= (ff_sel<=1)?    2 : 3;
    end
//.............................
// full
//.............................
4: if(FI_STB && FO_ACK)
    begin
     ff_sel      <=               ff_sel;
     ff_out_stb  <=                 1'b1;
```

```
      ff_state    <=                    4;
     end
    else if(FI_STB)
     begin
      ff_sel      <=         ff_sel + 1'b1;
      ff_out_stb  <=                1'b1;
      ff_state    <=                   4;
     end
    else if(FO_ACK)
     begin
      ff_sel      <=         ff_sel - 1'b1;
      ff_out_stb  <=                1'b1;
      ff_state    <= (ff_sel>=8)?    4 : 3;
     end
//.............................
default: ff_state <= 0;
//.............................
endcase
//-------------------------------------------------------------------------------------------
always@(posedge CLK)
 if(FI_STB) begin
                       ff_dat[4'h0] <= FI_DAT;
                       ff_dat[4'h1] <= ff_dat[4'h0];
                       ff_dat[4'h2] <= ff_dat[4'h1];
                       ff_dat[4'h3] <= ff_dat[4'h2];
                       ff_dat[4'h4] <= ff_dat[4'h3];
                       ff_dat[4'h5] <= ff_dat[4'h4];
                       ff_dat[4'h6] <= ff_dat[4'h5];
                       ff_dat[4'h7] <= ff_dat[4'h6];
                       ff_dat[4'h8] <= ff_dat[4'h7];
                       ff_dat[4'h9] <= ff_dat[4'h8];
                       ff_dat[4'hA] <= ff_dat[4'h9];
                       ff_dat[4'hB] <= ff_dat[4'hA];
                       ff_dat[4'hC] <= ff_dat[4'hB];
                       ff_dat[4'hD] <= ff_dat[4'hC];
                       ff_dat[4'hE] <= ff_dat[4'hD];
                       ff_dat[4'hF] <= ff_dat[4'hE];
                      end

//-------------------------------------------------------------------------------------------
assign                FO_DAT = ff_dat  [ff_sel];
//-------------------------------------------------------------------------------------------
endmodule
```

3. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu fifo.
4. Czy układ działa poprawnie? Sprawdzić z innymi wartościami liczb. Sprawdzić inne sposoby podawania danych:
   - kilka liczb w jednym testbench'u itp
   - wymuszenie związane z klawiaturę (fi_stb, fo_ack).
   Czy problem leży w module fifo, czy też w sposobie podawania danych? Jeśli program nie zachowuje się jak powinien, dokonać poprawek.
5. Wpisać poniższy program:

Plik testbench.v

```
module mytestbenchmodule();
reg CLK;
initial CLK  <= 0;
always #50  CLK <= ~CLK;

reg RST;
initial
begin
    RST <= 0;
    RST <= #100 1;
    RST <= #500 0;
end

STARTUP_FPGACORE glbl(.CLK(CLK), .GSR(RST));

reg        push_stb;
reg [31:0] push_dat;

wire [31:0] pop_dat;
reg         pop_stb;

initial
begin
    push_stb <= 0;
    push_dat <= 0;
    pop_stb <= 0;
    #1000;
    @(negedge CLK);
```

```
    push_stb <= 1;
    push_dat <= 17; @(negedge CLK);
    push_dat <= 18; @(negedge CLK);
    //      !!!!!!!!!!!!!!!! ....  please type some additional data here ... //
    //      !!!!!!!!!!!!!!!! ....  tutaj prosze wpisac dodatkowe dane ... //
    push_dat <= 23; @(negedge CLK);
    push_stb <= 0;  @(negedge CLK);
    #1000;
    @(negedge CLK);
    pop_stb <= 1;
end

stack s1
(
.CLK(CLK),
.RST(RST),
.PUSH_STB(push_stb),
.PUSH_DAT(push_dat),
.POP_STB(pop_stb),
.POP_DAT(pop_dat)
);

endmodule
```

Plik stack.v

```
module stack
(
input  wire        CLK,
input  wire        RST,
input  wire        PUSH_STB,
input  wire [31:0] PUSH_DAT,
input  wire        POP_STB,
output wire [31:0] POP_DAT
);
//-----------------------------------------------------------------------------
reg    [8:0] push_ptr;
//-----------------------------------------------------------------------------
reg    [8:0] pop_ptr;
//-----------------------------------------------------------------------------
always@(posedge CLK or posedge RST)
 if(RST)
  begin
    pop_ptr  <= 9'h1FF;
    push_ptr <= 9'h000;
  end
 else if(PUSH_STB)
  begin
   push_ptr <= push_ptr + 9'd1;
   pop_ptr  <= pop_ptr  + 9'd1;
  end
 else if(POP_STB)
  begin
   push_ptr <= push_ptr - 9'd1;
   pop_ptr  <= pop_ptr  - 9'd1;
  end
//-----------------------------------------------------------------------------
RAMB16_S36_S36
#(
.WRITE_MODE_A("WRITE_FIRST"), .WRITE_MODE_B("WRITE_FIRST")
)
stack
(
.CLKA(CLK), .ENA(1'b1), .WEA(PUSH_STB), .ADDRA(push_ptr), .DIA(PUSH_DAT), .DIPA(   4'd0),.DOA(       ),
.DOPA(       ), .SSRA(1'b0),
.CLKB(CLK), .ENB(1'b1), .WEB(    1'b0), .ADDRB( pop_ptr), .DIB(   32'b0), .DIPB(   4'h0),.DOB(POP_DAT),
.DOPB(       ), .SSRB(1'b0)
);
//-----------------------------------------------------------------------------------------------
endmodule
```

6. W opcji Design/Settings, zakładka "Verilog" dodać bibliotekę (Verilog Libraries) "ovi_unisim".
7. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu stack.
8. Czy układ działa poprawnie? Sprawdzić z innymi wartościami liczb. Sprawdzić inne sposoby podawania danych.
9. Kiedy moduł wystawia poprawne dane? Po jakim czasie (liczba taktów) of pojawienia się POP_STB na wyjściu pojawiają się te dane? Czy wystawiana jest informacja na temat potwierdzenia wystawianych danych?
10. Poprawić program tak, aby na wejściu i wyjściu występowały sygnały potwierdzenia (POP_ACK, PUSH_ACK).